

Seeing the Forest: A Taxonomy-Guided Analysis and Improvement of Malicious PyPI Package Detectors

Zibo Liu

Tianjin University
TianJin, China
zeber@tju.edu.cn

Junjie Wang

Tianjin University
TianJin, China
junjie.wang@tju.edu.cn

Sen Chen[✉]

Nankai University
TianJin, China
senchen@nankai.edu.cn

Abstract

Malicious packages pose a growing threat to open-source software ecosystems, yet existing detectors rely on fragmented and tool-specific behavior definitions, making it difficult to systematically characterize malicious intent or identify their detection blind spots. This paper presents an automated framework for constructing a fine-grained taxonomy of malicious behaviors in PyPI packages. We collect 11,362 real-world malicious packages from four public datasets and threat intelligence gathered from 12 mirror sources. Our framework combines four static analysis tools to locate sensitive code, leverages an LLM to reconstruct code context and extract structured behavior sequences, and applies card sorting and frequent subsequence mining to normalize behaviors and uncover recurring malicious intents. The framework achieves 95.6% accuracy in context reconstruction and 93.4% accuracy in behavior description, while a three-stage cross-review yields a Cohen's Kappa of 0.83. Based on 11,881 behavior sequences, we derive a taxonomy comprising seven major categories and 31 fine-grained behavior patterns. Using this taxonomy, we evaluate four existing static detectors and reveal substantial category-level blind spots that are obscured by aggregate metrics, particularly for code obfuscation and unauthorized system operations. We further use GuardDog as a case study and improve its recall for three weak behavior categories to above 90%. These results demonstrate that the proposed taxonomy provides a systematic basis for understanding malicious intent, evaluating detector coverage, and guiding the improvement of malicious package detection.

CCS Concepts

• Security and privacy → Malware and its mitigation.

Keywords

Software supply chain security; Malicious package detection; Malicious behavior analysis

ACM Reference Format:

Zibo Liu, Junjie Wang, and Sen Chen. 2026. Seeing the Forest: A Taxonomy-Guided Analysis and Improvement of Malicious PyPI Package Detectors. In *Proceedings of the 2nd International Workshop on Software Genomics (SWGeno '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3844178.3844648>



This work is licensed under a Creative Commons Attribution 4.0 International License. SWGeno '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3002-3/2026/10
<https://doi.org/10.1145/3844178.3844648>

1 Introduction

The widespread adoption of open-source software has improved development efficiency but has also created highly interconnected dependency ecosystems, increasing the risk of malicious code propagation [1]. Once introduced into a widely used package, malicious code can rapidly spread to downstream projects [2], causing data breaches, service disruption, and system compromise. PyPI is particularly exposed because of its large package population, low publishing barriers, and broad production adoption. The PyTorch torchtriton incident [3], in which attackers abused pip's index precedence to distribute a malicious package that exfiltrated sensitive user data, illustrates the potentially broad impact of a single compromised package. These risks make accurate malicious package detection essential to software supply chain security.

Numerous tools and techniques have therefore been developed to detect malicious packages. Rule-based approaches [4, 5] efficiently identify known malicious patterns, but depend on manually defined and continuously maintained rules, limiting their generalizability to unseen implementations. Machine-learning-based methods [6, 7] learn statistical differences between benign and malicious packages, but require representative labeled data and may degrade as attacker strategies evolve. More recent approaches increasingly analyze package behaviors and infer malicious intent from API usage, execution traces, or semantic representations [8–10]. However, their behavior representations are typically designed for individual detection pipelines. For example, DonAPI [8] embeds its API, sensitive-behavior, and classification layers into a specific workflow, while Cerebro [9] captures malicious semantics through the latent representations of a fine-tuned BERT model. Such representations are difficult to interpret, compare, reuse, or maintain across tools.

Despite this shift toward behavior- and intent-oriented detection, the field still lacks a comprehensive and standardized taxonomy of malicious behaviors. Existing datasets generally provide only coarse package-level labels [11, 12], while individual tools define behaviors according to their own detection objectives and technical implementations. Consequently, similar behaviors may be described at different granularities, distinct behaviors may be merged under the same label, and the relationships between code-level operations, observable behaviors, and high-level malicious intents remain unclear. Without a systematic view of the malicious behavior space, it is difficult to determine what existing tools actually cover, where their blind spots lie, and how their detection capabilities should be improved. A fine-grained and tool-independent taxonomy is therefore needed not only to characterize malicious packages, but also to provide a common basis for evaluating and improving existing detectors.

Constructing such a taxonomy presents three main challenges. **First, fine-grained labeled data are scarce.** Existing datasets provide package metadata or coarse labels but rarely identify malicious-code locations, behavior types, attack targets, and implementation techniques. **Second, malicious behaviors are highly heterogeneous [13].** The same intent may be realized through different APIs, code structures, and execution strategies, making cross-sample comparison difficult without a unified representation. **Third, fine-grained annotation is expensive to scale.** Manual analysis can produce reliable labels, but examining thousands of packages requires substantial expert effort, while automated methods must preserve sufficient semantic accuracy and consistency for taxonomy construction.

To address these challenges, we propose an automated framework for constructing a fine-grained taxonomy of malicious behaviors in PyPI packages. We collect 11,362 real-world malicious packages from four public datasets and threat intelligence gathered from 12 mirror sources. The framework first combines four static analysis tools, namely Bandit4Mal, PyPI Check, Aura, and GuardDog, to identify 167,846 sensitive code lines. It then uses an LLM to reconstruct the surrounding context, filter false-positive sensitive operations, locate malicious code fragments, and decompose each fragment into a structured behavior sequence describing its operations, targets, and implementation techniques. These descriptions are normalized using a behavior schema developed through card sorting [14]. Finally, frequent subsequence mining is applied across samples to identify recurring behavioral structures that reveal common malicious intents.

We validate the reliability of the framework at multiple stages. The LLM-based analysis achieves 95.6% accuracy in context reconstruction and 93.4% accuracy in behavior description. The resulting annotations are further examined through a three-stage cross-review process, yielding a Cohen’s Kappa of 0.83. Based on 11,881 extracted behavior sequences, we construct a taxonomy consisting of seven major categories and 31 fine-grained malicious behavior types.

The resulting taxonomy enables us to evaluate existing tools at the behavior-category level rather than relying solely on aggregate detection metrics. Our evaluation of four static tools shows that even when overall performance appears high, substantial blind spots remain in specific categories. The best-performing tools achieve up to 92.73% precision and 98.14% recall overall, but code obfuscation and unauthorized system operations remain particularly challenging. For example, GuardDog achieves only 59.79% recall and a 71.10% F1-score for code obfuscation, and 83.30% recall and an 86.97% F1-score for unauthorized system operations.

We further use GuardDog as a case study to examine how the taxonomy can guide detector improvement. Based on the identified weaknesses, we supplement detection rules for low-recall categories, including code obfuscation, remote control, and unauthorized system operations, while constraining overly broad rules that produce false positives. The enhanced rules raise recall for all three categories above 90%, with some reaching 100%. Nevertheless, the results also expose the inherent limitations of rule-based detection: expanding rule coverage can introduce false positives, whereas narrowly defined rules remain vulnerable to implementation variants. The taxonomy therefore serves both as an actionable

Table 1: Summary of the Collected Malicious Packages.

Source Type	Source	Reported	Available
Existing Datasets	Backstabber’s Knife	2,483	2,483
	MalOSS	263	263
	DataDog	1,618	1,618
	PyPI Malregistry	9,537	9,537
Intelligence Sites	Checkmarx	6,789	6,658
	Snyk.io	633	525
	Tianwen	904	431
	JFrog	145	138
	Sonatype	18	17
	Phylum	25	20
	Socket	7	6
	Independent Security Blog	36	34
Sum (before deduplication)		22,458	21,730
Total (deduplicated)		11,907	11,362

guide for improving existing tools and as a means of revealing the limitations of their underlying detection paradigms.

Overall, this study makes the following contributions:

- **A fine-grained dataset of malicious PyPI packages.** We collect 11,362 malicious packages and annotate their malicious-code locations, behavior types, attack targets, and implementation techniques, with a Cohen’s Kappa of 0.83.
- **An automated framework for malicious behavior analysis.** The framework combines static analysis, LLM-based semantic interpretation, and behavior sequence mining to extract and normalize malicious behaviors at scale.
- **A taxonomy for evaluating and improving detectors.** From 11,881 behavior sequences, we derive seven major categories and 31 behavior types, use them to reveal blind spots in four existing tools, and guide the improvement of GuardDog.

2 Approach

In this section, we introduce a malicious behavior analysis method based on code semantic understanding and behavioral sequence extraction, which comprises four stages as illustrated in Figure 1: (1) collection of malicious package datasets, (2) extraction of malicious code, (3) construction of behavioral sequences, and (4) classification of malicious behaviors.

2.1 Malicious Packages Collection

To support malicious behavior analysis, we first collect a large volume of malicious package data from two complementary sources: existing malicious package datasets and threat intelligence.

Collection from Existing Datasets. Prior analytical studies on malicious packages have curated a substantial number of samples. We searched Google Scholar for literature published between 2020 and 2026 using keywords such as “malicious package” and “malicious behavior.” We included peer-reviewed empirical studies that release or reference publicly accessible malicious package datasets, excluding purely survey papers, vulnerability-focused studies, and datasets no longer accessible. This process yielded 23 representative papers. We further screened these papers for publicly released PyPI-related datasets with usable package samples; the other papers targeted other ecosystems (e.g., npm, Maven), provided only aggregate statistics, or hosted unavailable datasets. Ultimately, four datasets met our criteria: Backstabber’s Knife Collection [11], MalOSS [12], DataDog [15], and PyPI Malregistry [13]. These datasets

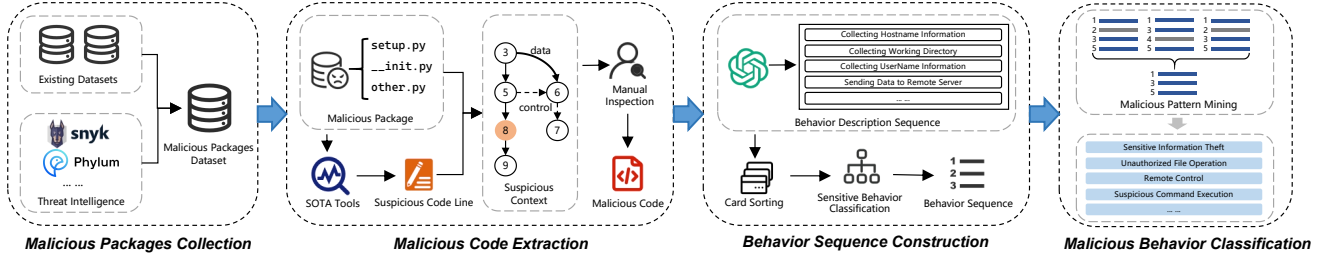


Figure 1: Malicious behavior analysis workflow.

are maintained as independent archives (e.g., on GitHub) that do not depend on the official PyPI registry, so all listed packages remain available regardless of subsequent official removal, as reflected by their equal Reported and Available counts in Table 1.

Collection Based on Threat Intelligence. Because existing datasets are updated infrequently, we augmented our collection with threat intelligence reported up to April 2026. We used Google with queries such as “malicious package” and “supply chain attack,” retrieving 4,821 web pages. We screened these for pages explicitly reporting PyPI malicious packages with identifiable names and version numbers, excluding purely informational articles, pages lacking specific package identifiers, and duplicates. This identified 8 usable intelligence sources (see Table 1), from which we manually extracted package names and version numbers. Since official registries typically remove malicious packages, we downloaded them from PyPI mirror sources during the synchronization window—mirrors replicate the official registry with a delay, so removed packages remain temporarily available. We utilized 12 mirror sources [16–27]. We verified that each downloaded package’s name and version matched those reported in the threat intelligence to avoid retrieving patched releases. Packages unavailable within the synchronization window were excluded, accounting for the 545 unavailable packages in Table 1.

After merging both sources, we deduplicated by package name and version number, yielding 11,907 unique packages. The sources overlap substantially (notably PyPI Malregistry and Checkmarx), resulting in 10,551 removed duplicates. Among the unique packages, 545 from threat intelligence sources could not be retrieved from any mirror within the synchronization window and were excluded, leaving a final dataset of 11,362 malicious packages (Table 1).

2.2 Malicious Code Extraction

Existing datasets typically lack annotations for the locations of malicious code. We therefore adopt a three-step approach: locating sensitive operations with existing tools, reconstructing the code context with an LLM, and manually verifying the results.

Sensitive Operation Localization. We surveyed mainstream malicious package detection tools (Table 2). Based on availability and the ability to output suspicious code locations, we selected Bandit4Mal [5], PyPI Check [28], Aura [29], and GuardDog [4]. We applied these four tools to scan the malicious packages in our dataset and aggregated the sensitive code lines identified by each tool, yielding a total of 167,846 sensitive code lines. As the four tools employ complementary rule sets, their outputs were merged by file path and line number, unifying overlapping entries into a

single record. Although the aggregated set contains false positives, they are filtered out by subsequent manual verification.

Code Context Reconstruction. We then employed GPT-4o-mini to reconstruct the code context surrounding each identified sensitive operation. Specifically, we fed the source code file along with the list of sensitive code lines as input to the LLM, which extracted the malicious code fragments by analyzing control flow, data flow, and code semantics. The core prompt template is as follows:

Please act as a malicious code analysis expert. I will send you {code file} and {sensitive code list} involving sensitive operations extracted from it. Please extract the malicious code that can reflect complete malicious behaviors from the file.

1. Conduct analysis by combining the control flow, data flow information and code semantics of the code.
2. Focus on whether the code logic associated with sensitive operations contains malicious code.
3. If there is no malicious code in the file content, return an empty result.

We configured the decoder with temperature=0, top_p=1, and a fixed decoding seed to minimize output variation across runs via greedy decoding. We further constrained the output to a structured JSON format specifying the extracted code fragment and its location, ensuring parseable and consistent results.

Manual Verification. We manually verified all LLM reconstructed fragments. Two authors with over 3 years of security research experience independently reviewed every fragment. A fragment was retained only if it satisfied two criteria: (1) the code exhibits clear malicious intent rather than benign API use; (2) the fragment boundaries are appropriate, encompassing all statements necessary for the malicious behavior without excessive unrelated code. When the two reviewers disagreed on either criterion, a third author served as an arbiter and the final decision was determined by majority vote. In total, the LLM produced 17,947 candidate fragments; after verification, 6066 were removed as false positives, yielding 11,881 confirmed malicious code fragments. Among all candidates, 1,673 were initially flagged as disputed and resolved through arbitration. The Cohen’s Kappa coefficient between the two reviewers was 0.9068, indicating strong inter-rater agreement. Additionally, since rule-based detection tools may fail to flag malicious operations in heavily obfuscated or dynamically generated code, some packages may yield no candidate fragments through the automated pipeline. By construction, every package in our dataset was reported as malicious; therefore, for the 116 packages where

Table 2: Malicious package detection tools.

Tool	Method	Availability	Output
PyPI Check [28]	Rule Matching	Open-source	Yes
Bandit4Mal [5]	Rule Matching	Open-source	Yes
MalOSS [12]	Static & Dynamic	Open-source	No
PPD [7]	Static Analysis	Closed-source	No
Aura [29]	Static & Rules	Open-source	Yes
AMALFI [6]	Machine Learning	Results only	No
GuardDog [4]	Static & Rules	Open-source	Yes
MPHunter [30]	Machine Learning	Open-source	No
MalWuKong [31]	Static Analysis	Partial	No
SAP [32]	Machine Learning	Open-source	No
OSCAR [33]	Dynamic Analysis	Closed-source	No
DONAPI [8]	Static & Dynamic	Closed-source	No
EA4MP [34]	Machine Learning	Open-source	No
Cerebro [9]	Machine Learning	Closed-source	No
PyPIGuard [35]	Machine Learning	Closed-source	No

no fragments were extracted, we performed manual code review to identify missed malicious code. Through this process, 38 additional malicious code fragments were identified and incorporated into the final set, mitigating the false-negative risk inherent in the tool-based localization.

2.3 Behavior Sequence Construction

In this section, we construct behavior sequences from the extracted malicious code snippets using LLM-based semantic analysis. To address LLM output variation, we suppress variation across repeated runs through a greedy decoder configuration and normalize residual lexical variation via behavior label induction, yielding structured behavior sequences.

LLM-Generated Behavior Description Sequences. We employed GPT-4o-mini to decompose the behaviors of malicious code, with temperature=0, top_p=1, and a fixed decoding seed to suppress variation across repeated runs. The model outputs a structured JSON array where each element carries a behavior description and its supporting code location. The prompt requires the LLM to reconstruct the code execution order through control-flow and data-flow analysis and decompose behaviors accordingly, ensuring each behavior reflects complete and atomic semantics with standardized expressions. The complete prompt additionally specifies the ordering conventions derived from static analysis: all conditional branches are preserved in control-flow order, loop bodies are recorded once without unrolling, and asynchronous behaviors are ordered by inferred trigger time. The JSON schema and core template are shown below:

Please act as a malicious code analysis expert. Please help me analyze the malicious behavior in the {malicious code}, decomposing malicious code into a combination of several {sensitive behaviors}.
1. Each sensitive operation can consist of one or more lines of code, but it must reflect complete and atomic code semantics.
2. Descriptions of sensitive behaviors need to be clear and concise, using standardized expressions.
3. When generating sensitive behavior sequences, the execution order of the code should be preserved as much as possible through control-flow and data-flow analysis.

Table 3: Sensitive behavior classification results.

Category	Sensitive Behaviors
Information Collection	Obtain clipboard info, obtain online account info, obtain location info, obtain user input, obtain license info, obtain local repository log info, obtain process list, obtain user password, obtain environment variable info, obtain network info, obtain hardware info, obtain browser info, obtain system info, obtain database info, obtain file content, obtain directory info, obtain file attributes, obtain subnet host info, obtain camera data, obtain registry info, obtain identity info, obtain software installation list, obtain screenshot, obtain microphone audio
Network	Create network connection, obtain remote command, modify online info, send data, download remote data
File	Create directory, modify file attributes, modify clipboard content, modify file output stream, modify file name, delete local file, create file, modify database info, delete database content, move file, modify registry, modify file content,
System	Shutdown system, stop process, disable system protection, launch application, inject dynamic library, create user, create terminal, create scheduled task, mount device, uninstall component, install component
Data Processing	Extract data, data packaging, construct file path, construct network request packet, construct link, store payload, command construction, data encoding, data decoding, compress file, decompress data, serialize data, deserialize data, code obfuscation, code deobfuscation
Execution	Import module, create new thread, download and execute file, execute local file, extract data from image and execute, command execution

Through this process, each code snippet was decomposed into a sequence of sensitive behavior descriptions ordered by the execution sequence reconstructed through control-flow and data-flow analysis, yielding 11,881 sensitive behavior description sequences.

Behavior Label Induction. We utilized card sorting [14] to inductively summarize behavior descriptions into structured labels. After merging and deduplicating all descriptions (yielding 12,888 unique descriptions), we equally divided them into three groups of 4,296. Three authors independently performed card sorting on their assigned group, distilling descriptions into concise behavior cards; if a matching card already existed, the description was assigned to it. This yielded 133 initial cards, which the three authors collaboratively deduplicated based on synonymy and semantic similarity, ultimately producing 71 unique behavior cards. These cards were further grouped into 6 top-level categories, forming the sensitive behavior classification framework (Table 3).

To ensure label accuracy, we designed a three-stage cross-review mechanism. For each group, an author not involved in the initial annotation independently verified the cards; disagreements were resolved by a fourth author acting as arbiter, with the final decision determined by majority vote. Of the 12,888 descriptions, 2,142 were flagged as disputed and 863 underwent category adjustments. The Cohen’s Kappa coefficient between reviewers reached 0.8338, suggesting high consistency among reviewers. Replacing raw descriptions with the standardized labels yielded structured behavior sequences for all 11,881 malicious code snippets.

2.4 Malicious Behavior Classification

With the structured behavior sequences constructed, we further mine malicious behavior patterns from them and classify these patterns into distinct malicious behavior categories. This process

Table 4: Malicious behavior patterns.

Category	ID	Sequence	Support
Sensitive Information	A1	Obtain network info → Data Packaging → Send data	1,137
	A2	Obtain identity info → Data Packaging → Send data	964
	A3	Obtain directory info → Data Packaging → Send data	392
	A4	Obtain system info → Data Packaging → Send data	280
	A5	Obtain file content → Data Packaging → Send data	211
	A6	Obtain env variable info → Data Packaging → Send data	176
	A7	Obtain browser info → Data Packaging → Send data	146
	A8	Obtain online account info → Data Packaging → Send data	83
	A9	Obtain file attributes → Data Packaging → Send data	56
	A10	Obtain hardware info → Data Packaging → Send data	52
	A11	Obtain process list → Data Packaging → Send data	36
	A12	Obtain screenshot → Data Packaging → Send data	34
	A13	Obtain database info → Data Packaging → Send data	24
	A14	Obtain clipboard info → Data Packaging → Send data	11
Unauthorized File Operation	B1	Delete local file	258
	B2	Download remote data → Modify file content	44
	B3	Modify registry	15
Remote Control	C1	Create network connection → Modify file output stream → Create network connection	44
	C2	Create network connection → Modify file output stream → Create terminal	31
Suspicious Command Execution	D1	Data Decoding → Command execution	8,008
Local File Execution	E1	Download remote data → Modify file attributes → Execute Local file	802
	E2	Modify file content → Execute Local file	6
	E3	Obtain file content → Execute Local file	5
Unauthorized System Operation	F1	Install component	348
	F2	Launch application	168
	F3	Stop process	79
	F4	Disable system protection	41
	F5	Create scheduled task	34
	F6	Shutdown system	14
Code Obfuscation	G1	Code Obfuscation	3,159
	G2	Code Deobfuscation	640

consists of two steps: frequent pattern mining to extract recurring behavior subsequences, and malicious behavior categorization to assign semantic labels to the discovered patterns.

Frequent Pattern Mining. With the structured behavior sequences extracted, we employ the PrefixSpan algorithm [36] to mine malicious patterns and classify malicious behaviors. Taking the behavior sequence set as input, we set the minimum support threshold to 5; this threshold strikes a balance between filtering out spurious, infrequent patterns that may arise from noise and retaining meaningful malicious patterns that occur with sufficient regularity to be considered representative attack behaviors. PrefixSpan recursively discovers frequent subsequences satisfying the support threshold by constructing projected databases. To eliminate redundancy, the resulting frequent subsequences are sorted by length in ascending order; a short sequence manually annotated as a complete malicious behavior pattern is incorporated into the valid pattern set, and any longer sequence containing it is discarded. This process yielded 31 non-redundant malicious behavior patterns.

Malicious Behavior Categorization. Two authors independently classified the 31 patterns into malicious behavior categories; disagreements were resolved by a third author through discussion and majority vote. We retained six categories from prior work [9, 13, 31]—suspicious command execution, sensitive information theft, remote control, unauthorized file operation, local file execution, and code obfuscation—and introduced “unauthorized system operation” for behaviors involving privileged operations (e.g., modifying

system configurations, starting/stopping critical services) not covered by existing labels. The seven core malicious behavior types are defined as follows:

- **Sensitive Information Theft:** Attackers obtain sensitive information, package the collected data, and exfiltrate it to remote servers.
- **Unauthorized File Operation:** Attackers modify, delete, or move sensitive files without user permission, threatening normal system operation.
- **Remote Control:** Attackers establish network-based control channels to issue commands on the victim host and receive execution results.
- **Suspicious Command Execution:** Attackers decode data to obtain or construct malicious commands and execute them on the victim host.
- **Local File Execution:** Attackers download or embed malicious files and execute them on the victim host.
- **Unauthorized System Operation:** Attackers perform privileged operations (e.g., installing components, stopping processes, shutting down the system) without authorization.
- **Code Obfuscation:** Attackers disguise malicious code via variable renaming, encoding, or encryption to evade detection and hinder reverse analysis.

The extracted malicious behavior patterns and their support values are shown in Table 4.

2.5 Behavior Sequence Evaluation

To assess the accuracy of behavior sequence construction, we evaluated two critical steps in the pipeline: LLM-based code context completion and sensitive behavior description generation.

Accuracy of Code Context Completion. To evaluate the LLM’s performance in completing code contexts, we randomly

Table 5: Distribution of malicious behaviors.

Category	Package	Ratio
Suspicious Command Execution	7,486	59.30%
Code Obfuscation	2,390	18.95%
Sensitive Information Theft	1,271	10.07%
Unauthorized File Operation	240	1.90%
Local File Execution	651	5.16%
Remote Control	60	0.48%
Unauthorized System Operation	530	4.20%
Total	12,628	100.00%

selected 1000 malicious code samples. We then compared the manually extracted malicious code with the code completed by the LLM. Due to the inherent variability and potential subjective bias in analyzing malicious code, achieving exact string-level consistency is impractical. Therefore, we adopted a semantic evaluation criterion: the LLM-completed code context is considered accurate if it encompasses all the constituent behaviors required to form the malicious action. Based on this criterion, 956 out of the 1000 extracted behavior sequences were consistent with the ground truth, yielding an accuracy rate of 95.60%.

Accuracy of Sensitive Behavior Description. To evaluate the LLM’s capability in behavior sequence decomposition and sensitive behavior description generation, we applied it to 1000 malicious code files to generate sensitive behavior description sequences. Concurrently, we manually decomposed the behaviors of these files to construct a ground truth, following the same inferred-execution-order convention described above so that the LLM and the annotators shared an identical notion of sequence order. The evaluation metric was whether the LLM-generated descriptions could produce sensitive behavior sequences identical to those derived from manual annotation. The results shows that 934 out of the 1000 behavior sequences matched, resulting in an accuracy rate of 93.40%.

3 Empirical Study

In this section, we formulate three research questions to evaluate the effectiveness of the malicious behavior classification model.

RQ1: What is the distribution of malicious behaviors and what characteristics does each type exhibit? **RQ2:** How effective are current detection tools in identifying each category of malicious behavior and what factors contribute to their suboptimal performance? **RQ3:** Based on the proposed malicious behavior classification model, how can existing tools be improved, and to what extent does the improvement enhance their detection performance?

3.1 RQ1: Distribution and Patterns of Malicious Behaviors

We analyze the distribution of each malicious behavior type at the package level in the PyPI ecosystem. Since a malicious package may contain multiple behavior sequences of different types, we retained and separately counted multi-category packages in all their associated categories.

As shown in Table 5, “Suspicious Command Execution” dominates at 59.30%, primarily because its implementation is simplest—invoking built-in APIs suffices—and the achievable attack capabilities are highly diverse, making it the most cost-effective choice. “Code Obfuscation” ranks second (18.95%) because attackers typically combine obfuscation with other techniques to evade detection,

Table 6: Common malicious behavior patterns.

Malicious Behavior Sequence	Support
Data Decoding → Command execution	8,008
Code Obfuscation	3,159
Obtain network info → Data Packaging → Send data	1,137
Obtain identity info → Data Packaging → Send data	964
Download remote data → Modify file attributes → Execute local file	802
Code Deobfuscation	640
Install component	348
Obtain directory info → Data Packaging → Send data	392
Obtain system info → Data Packaging → Send data	280
Delete local file	258
Total	15,788

forming the prevalent “decode-then-execute” pattern. The remaining categories each occupy less than 10.07%, as they either require complex setup or primarily serve as auxiliary steps in multi-stage chains rather than independent attack vectors.

To investigate the occurrence patterns of attack behavior sequences, we use support as the core metric. As shown in Table 6, “Data Decoding → Command execution” is the most mainstream pattern (support 8,008), as encoding bypasses static detection and the subsequent execution delivers the payload—together forming a self-contained, low-effort attack chain. “Code Obfuscation” alone ranks second (3,159), typically serving as a preparatory wrapper for other malicious logic. The top 10 sequences account for over 91% of total support, revealing that attackers strongly prefer concise, proven patterns over complex multi-step chains, as simpler patterns are easier to construct, harder to detect, and sufficient for most attack goals.

3.2 RQ2: Evaluation of SOTA Tools

To evaluate the detection effectiveness of existing tools, we construct a balanced dataset comprising all 12,628 malicious packages as positive samples and an equal number of benign packages as negative samples. The benign packages are randomly selected from PyPI packages with over 100 downloads in the past year. We apply four representative tools—Bandit4Mal, PyPI Check, Aura, and GuardDog for each malicious behavior category. Results are shown in Table 7. GuardDog achieves the best overall performance with accuracy 76%–95% and precision 88%–93%, while the other three tools exhibit notable deficiencies. We analyze the root causes by correlating with the malicious behavior sequences identified in RQ1:

Bandit4Mal relies on static pattern matching for API calls and string fragments. It cannot detect the “Data Decoding → Command execution” pattern where payloads are decoded at runtime, yielding only 27.77% recall on Suspicious Cmd Exec. **Aura** adds lightweight scoring on top of static matching, but without data-flow analysis it misses the “Data Decoding → Command execution” pattern at 25.73% recall. **PyPI Check** uses YARA rules targeting import patterns with precision 81%–90%, but misses alternative channels like `requests/urllib` in exfiltration and lacks rules for “Download → Execute local file” and “Install component”, leading to recall of 48%–68% on several categories. **GuardDog** combines `semgrep` rules with taint analysis, achieving the strongest results overall. Its remaining gaps are: on “Code Obfuscation” recall is 59.79% since its rules target eval-based execution rather than obfuscation alone; on “Install component” recall is 83.30% due to similarity between

Table 7: Comparison of tool detection results.

Tool	Category	Acc.	Prec.	Recall	F1
Bandit4Mal	Sensitive Info Theft	60.62%	56.01%	99.37%	71.64%
	Unauthorized File Op	57.62%	54.18%	100.00%	70.28%
	Remote Control	56.78%	54.05%	100.00%	70.18%
	Suspicious Cmd Exec	23.86%	25.82%	27.77%	26.76%
	Local File Exec	55.92%	53.48%	92.17%	67.68%
	Unauthorized Sys Op	58.58%	54.75%	98.87%	70.48%
	Code Obfuscation	58.63%	54.96%	96.32%	69.98%
PyPI Check	Sensitive Info Theft	65.17%	83.59%	59.76%	69.70%
	Unauthorized File Op	58.92%	80.56%	48.33%	60.42%
	Remote Control	79.79%	90.20%	76.67%	82.88%
	Suspicious Cmd Exec	88.05%	89.63%	92.95%	91.26%
	Local File Exec	71.24%	85.38%	68.20%	75.83%
	Unauthorized Sys Op	68.94%	85.68%	63.21%	72.75%
	Code Obfuscation	82.48%	89.31%	83.93%	86.53%
Aura	Sensitive Info Theft	66.98%	62.57%	84.88%	72.03%
	Unauthorized File Op	68.06%	63.22%	86.67%	73.11%
	Remote Control	71.43%	65.85%	90.00%	76.06%
	Suspicious Cmd Exec	38.39%	34.56%	25.73%	29.50%
	Local File Exec	64.31%	60.73%	81.26%	69.51%
	Unauthorized Sys Op	70.57%	64.34%	92.26%	75.81%
	Code Obfuscation	73.86%	66.62%	95.90%	78.62%
GuardDog	Sensitive Info Theft	91.88%	91.57%	92.15%	91.86%
	Unauthorized File Op	88.87%	92.63%	84.45%	88.35%
	Remote Control	88.98%	92.59%	84.75%	88.50%
	Suspicious Cmd Exec	94.98%	92.30%	98.14%	95.13%
	Local File Exec	90.04%	92.73%	86.84%	89.69%
	Unauthorized Sys Op	87.63%	90.99%	83.30%	86.97%
	Code Obfuscation	76.32%	87.70%	59.79%	71.10%

malicious and benign installations; on “Download → Execute local file”, indirect execution via ctypes or obfuscated Popen falls outside existing taint rules.

3.3 RQ3: Enhancement of SOTA Tools

To evaluate the improvement, we randomly select 2,000 malicious packages from our classified dataset as positive samples and an equal number of benign packages from PyPI packages with over 100 downloads in the past year as negative samples. We improve GuardDog in two stages based on the behavior sequences identified in RQ1: rule supplementation to boost recall for underdetected sequences, and rule optimization to reduce false positives by distinguishing malicious chains from benign isolated actions.

For rule supplementation, we target three low-recall sequences: (1) “Code Obfuscation”—existing rules miss encoding wrappers like base64/hex, so we add rules flagging common encoding functions as standalone wrappers; (2) “Download → Modify file attributes → Execute local file”—we introduce taint tracking from download sources through file modification to execution sinks; (3) “Install component”—we add rules detecting installation followed by suspicious post-install behavior like network requests or command execution.

For rule optimization, we exploit that malicious sequences form complete chains while benign triggers are isolated: (1) the code-execution rule is restricted to non-literal arguments, since dynamically constructed commands indicate the “Data Decoding → Command execution” pattern; (2) the api-obfuscation rule is narrowed from any getattr/exec to high-risk methods exec, eval, system, and Popen; (3) the shady-links rule requires flagged URLs to appear in data transmission or download contexts.

After improvement, the most significant gains are on previously weak categories: recall for code obfuscation rises from 64.10%

Table 8: GuardDog performance before and after improvement.

Category	Stage	Acc.	Prec.	Recall	F1
Sensitive Info Theft	Before	92.56%	92.66%	92.31%	92.49%
	After	95.88%	95.20%	96.63%	95.91%
Suspicious Cmd Exec	Before	95.29%	92.60%	98.41%	95.42%
	After	96.26%	94.34%	98.43%	96.34%
Local File Exec	Before	90.48%	90.48%	90.48%	90.48%
	After	95.67%	94.62%	96.85%	95.72%
Unauthorized Sys Op	Before	90.14%	93.68%	85.58%	89.45%
	After	97.25%	94.78%	100.00%	97.32%
Unauthorized File Op	Before	91.21%	93.18%	89.13%	91.11%
	After	93.48%	91.67%	95.65%	93.62%
Remote Control	Before	83.33%	81.25%	86.67%	83.87%
	After	90.00%	87.50%	93.33%	90.32%
Code Obfuscation	Before	78.37%	88.18%	64.10%	74.23%
	After	91.08%	91.77%	90.25%	91.00%

to 90.25%, remote control from 86.67% to 93.33%, and unauthorized system operation from 85.58% to 100%. Accuracy improves across all categories, with the largest increase on code obfuscation (78.37%→91.08%) and remote control (83.33%→90.00%). Overall precision improves from 90% to 94%, confirming that sequence-aware supplementation and chain-structure-driven optimization effectively enhance both coverage and accuracy.

4 Threats to Validity

Internal validity. The LLM-based pipeline may introduce errors despite deterministic decoding and cross-review with Cohen’s Kappa of 0.91 for code extraction and 0.83 for behavior labeling. The PrefixSpan support threshold of 5 is a parameter choice; adjusting it may yield different patterns. Multi-category packages counted in all associated categories may inflate proportional statistics. The evaluation uses a balanced benign-to-malicious ratio that may not reflect real-world class imbalance. **External validity.** Our study is confined to PyPI; whether the classification and improvements generalize to npm or Maven remains to be validated, as different language and package management characteristics may yield different malicious patterns. The rule improvements are also specific to GuardDog’s architecture. **Construct validity.** Packages with a “loader + payload” architecture dynamically fetch payloads at runtime, so our static pipeline only captures first-stage behavior. Heavily obfuscated or dynamically generated code may evade tool-based localization, potentially undercounting certain behavior categories.

5 Related Work

Malicious behavior analysis. Ohm et al. [11] pioneered the Backstabber’s Knife Collection, modeling malicious code injection paths and behavior trigger mechanisms. Duan et al. [12] proposed the MalOSS framework, combining metadata, static, and dynamic rules to uncover previously undetected malicious packages. Guo et al. [13] conducted an empirical study analyzing the lifecycle and type-specific characteristics of malicious packages, revealing that malicious behaviors are highly heterogeneous. These studies provide the data foundation for this work, yet their coarse-grained labels lack annotations on malicious-code locations, behavior types, and attack targets.

Behavior-based modeling. Huang et al. [8] proposed Don-API, which combines static screening and dynamic analysis to construct a three-layer model (API sequences, sensitive behavior sequences, and malicious behavior classification); however, its behavior definitions are bound to a single detection pipeline, making them neither reusable nor comparable across tools. Zhang et al. [9] constructed malicious code behavior sequences through feature extraction and leveraged a fine-tuned BERT model, yet its latent representations remain opaque to downstream rule maintenance. Zahan et al. [10] proposed the SocketAI workflow using CodeQL for static pre-filtering and GPT models for deep analysis, but still lacks a standardized, fine-grained formalization of malicious behaviors.

6 Conclusion

This paper presents an automated framework for constructing a fine-grained taxonomy of malicious behaviors in PyPI packages. By analyzing 11,362 malicious packages, we derive 11,881 behavior sequences and organize them into seven major categories and 31 behavior types. The taxonomy provides a unified basis for understanding malicious intent and evaluating detector coverage. Our evaluation of four static tools reveals category-level blind spots that are obscured by aggregate metrics. Using GuardDog as a case study, we further demonstrate that the taxonomy can guide detector improvement, while also exposing the inherent trade-off between behavior coverage and false positives in rule-based detection.

Acknowledgments

This work was supported by the Tianjin Major Science and Technology Special Project (Grant No. 25ZXSFN00140).

References

- [1] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. doi:10.1109/SP46215.2023.10179304
- [2] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 672–684. doi:10.1145/3510003.3510142
- [3] PyTorch Foundation. 2022. Compromised PyTorch-nightly Dependency Chain between December 25th and 30th, 2022. <https://pytorch.org/blog/compromised-nightly-dependency/>.
- [4] DataDog. 2022. GuardDog. <https://github.com/DataDog/guarddog>.
- [5] Bandit4Mal. 2020. Bandit4Mal. <https://github.com/lyvd/bandit4mal>.
- [6] Adriana Seifia and Max Schäfer. 2022. Practical Automated Detection of Malicious npm Packages. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 1681–1692. doi:10.1145/3510003.3510104
- [7] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 606–613. doi:10.1109/TrustCom53373.2021.00091
- [8] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qian-chong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 3765–3782. <https://www.usenix.org/conference/usenixsecurity24/presentation/huang-cheng>
- [9] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruiqi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 104 (April 2025), 28 pages. doi:10.1145/3705304
- [10] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2025. Leveraging Large Language Models to Detect npm Malicious Packages. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) (ICSE '25). IEEE Press, 2625–2637. doi:10.1109/ICSE55347.2025.00146
- [11] Wentao Liang, Xiang Ling, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 307–318. doi:10.1109/ASE56229.2023.00085
- [12] Ruian Duan, Omar Alrawi, R. P. Kasturi, et al. 2020. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. *arXiv preprint arXiv:2002.01139* (2020).
- [13] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2024. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) (ASE '23). IEEE Press, 166–177. doi:10.1109/ASE56229.2023.00135
- [14] Wikipedia. 2026. Card Sorting. https://en.wikipedia.org/wiki/Card_sorting.
- [15] DataDog. 2022. Malicious Software Packages Dataset. <https://github.com/DataDog/malicious-software-packages-dataset>.
- [16] Tsinghua University. 2026. PyPI Mirror of Tsinghua University. <https://pypi.tuna.tsinghua.edu.cn/simple/>.
- [17] Beijing Foreign Studies University. 2026. PyPI Mirror of BFSU. <https://mirrors.bfsu.edu.cn/pypi/web/simple/>.
- [18] NetEase Company. 2026. PyPI Mirror of NetEase Company. <https://mirrors.163.com/pypi/simple/>.
- [19] Alibaba Company. 2026. PyPI Mirror of Alibaba Company. <https://mirrors.aliyun.com/pypi/simple/>.
- [20] Tencent Company. 2026. PyPI Mirror of Tencent Company. <https://mirrors.cloud.tencent.com/pypi/simple/>.
- [21] Southern University of Science and Technology. 2026. PyPI Mirror of SUSTech. <https://mirrors.sustech.edu.cn/pypi/simple/>.
- [22] Kakao. 2026. PyPI Mirror of Kakao. <http://mirror.kakao.com/pypi/simple/>.
- [23] RStudio. 2026. PyPI Mirror of RStudio. <https://packagemanager.rstudio.com/pypi/latest/simple/>.
- [24] Neusoft. 2026. PyPI Mirror of Neusoft. <https://mirrors.neusoft.edu.cn/pypi/web/simple/>.
- [25] Nanyang Institute of Technology. 2026. PyPI Mirror of NYIST. <https://mirror.nyist.edu.cn/pypi/web/simple/>.
- [26] Peking University. 2026. PyPI Mirror of PKU. <https://mirrors.pku.edu.cn/pypi/web/simple/>.
- [27] University of Science and Technology of China. 2026. PyPI Mirror of USTC. <https://mirrors.ustc.edu.cn/pypi/simple/>.
- [28] Warehouse. 2019. PyPI Check. <https://github.com/pypi/warehouse>.
- [29] Source.AI. 2021. Aura. <https://github.com/SourceCode-AI/aura>.
- [30] Wentao Liang, Xiang Ling, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 307–318. doi:10.1109/ASE56229.2023.00085
- [31] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2023. MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1993–2005. doi:10.1109/ASE56229.2023.00073
- [32] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference* (Austin, TX, USA) (ACSAC '23). Association for Computing Machinery, New York, NY, USA, 71–82. doi:10.1145/3627106.3627138
- [33] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanhao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1990–2001. doi:10.1145/3691620.3695262
- [34] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1159–1170.
- [35] Tahir Iqbal, Guowei Wu, Zahid Iqbal, Muhammad Bilal Mahmood, Amreen Shafique, and Wenbo Guo. 2025. PypiGuard: A novel meta-learning approach for enhanced malicious package detection in PyPI through static-dynamic feature fusion. *J. Inf. Secur. Appl.* 90, C (May 2025), 15 pages. doi:10.1016/j.jisa.2025.104032
- [36] Jian Pei, Jiawei Han, B. Mortazavi-Asl, H. Pinto, Qiming Chen, U. Dayal, and Mei-Chun Hsu. 2001. PrefixSpan: mining sequential patterns efficiently by prefix-projected pattern growth. In *Proceedings 17th International Conference on Data Engineering*. 215–224. doi:10.1109/ICDE.2001.914830

Received 2026-07-27; accepted 2026-08-21